An Outline of a C++ Numbers Technical Specification

ISO/IEC JTC1 SC22 WG21 P0101R0 - 2015-09-27

Lawrence Crowl, Lawrence@Crowl.org

Introduction

ISO SC22/WG21/SG6 is the numerics study group of the C++ standards committee. SG6 has been working towards the definition of a set of number types and associated functions that enable

- the interchange of additional numeric values between software components with different authors,
- the production of software that is less vulnerable (if not immune) to overflow and rounding errors and
- the efficient and effective implementation of additional numeric types.

Design Principles

- Provide a consistent vocabulary.
- Expose sound "building-block" abstractions.
- Provide a mechanism for conversion that does not require n^2 operations or coordination between independent developers.
- Strive for efficient parameter passing. Pass types known to be primitive or very small by value. Pass types known to be indirect or very large by reference.
- Match aliasing behavior to the users. Handle potential parameter aliasing internally if the type is expected to be widely used. If the type serves mostly as a narrow-audience implementation tool, handle aliasing externally.
- Prefer run-time efficiency over compile-time efficiency.
- Only define functions to be constexpr if we need them constexpr today. It is easy to add constexpr later, but impossible to remove it.
- Prefer general-use types that can detect and handle exceptional conditions.

Built-in Types

We add or reference built-in types when necessary.

Decimal Floating Point

Decimal floating-point types already exist. We anticipate some refinement of them. See N3871 Proposal to Add Decimal Floating Point Support to C++.

Parametric Aliases

Computing the needed size of a type need support. See P0102R0 C++ Parametric Number Type Aliases.

Other Built-in Types

There has been discussion of the following types, but as yet no papers.

- non-modular unsigned integers
- binary floating-point intervals
- binary floating-point rationals
- binary log representation of reals

Utilities

The implementation of numeric types will often have common implementation components. It would be nice to share them.

Overflow-Detecting Arithmetic

Detecting overflow has existing hardware support that is not available to programmers. See P0103R0 Overflow-Detecting and Double-Wide Arithmetic Operations.

Double-Wide Arithmetic

Multi-word operations are most efficient when hardware-supported double-wide operations are available to programmers. See P0103R0 Overflow-Detecting and Double-Wide Arithmetic Operations.

Multiprecision Arithmetic

Multi-word operations and types are commonly needed. See P0104R0 Multi-Word Integer Operations and Types.

Other Utilities

We need some bitwise utilities along the lines of N3864 A constexpr bitwise operations library for C++.

While not strictly related to numbers, a library for units of measure would be welcome.

Rounding and Overflow

The primary problem with existing C++ number types is the very poor control over overflow and rounding. We need to put such control in the hands of programmers. See P0105R0 Rounding and Overflow in C++.

Unbounded Types

Unbounded types allocate memory as necessary to maintain a representation of true operation results. We may need an allocator interface for them.

Unbounded Integer

The classic bignum has been too long in coming to C++. See N4038 Proposal for Unbounded-Precision Integer Types.

Can we know highest bit set in integers?

Unbounded Rational

Unbound rational numbers is useful in computational geometry. See N3611 A Rational Number Library for C++.

Rational numbers can grow pretty quickly without reduction, but reduction is expensive. The tradeoff could be managed either by checking the size or by adding a count of operations since last reduction.

Other Unbounded Types

We do not yet have a paper for an unbound binary-floating point. It would be useful on its own and as a generalization of both fixed-bound floating-point and fixed-bound fixed-point. Rounding would still take place with unbound binary-floating point, but only on division and only within normal error bounds. This generally means a doubling in representation size after division. An explicit resolution reduction would be desireable.

Bounded Types

Bounds are specified in number of data bits, i.e. ignoring sign.

Bounded Integers

Integers are bound by their range. They can overflow. See P0106R0 C++ Binary Fixed-Point Arithmetic.

Bounded Fixed-Point Types

Integers are bound by their range and resolution. They can both overflow and require rounding. See P0106R0 C++ Binary Fixed-Point Arithmetic.

Other Bounded Types

We have discussed, but do not have papers for:

- A binary rational numberis bound by the range of the numerator and the range of the denominator. It has both overflow and rounding. Each operation would be exact until the representation size is reached, and then would approximate using a near representation. has overflow and rounding
- A bounded decimal fixed-point would be much like a bounded binary fixed-point.

Conversion

Conversion is not a new problem. See N1879 A proposal to add a general purpose ranged-checked numeric_cast<> (Revision 1). However, many new number types requires a practical conversion algorithm. Using converting through intermedate unbounded types provides that algorithm. See P0105R0 Rounding and Overflow in C++.